NAME: NITHIYASRI S
REGISTER NUMBER: 192572082
DEPARTMENT: CSE WITH AI

SUBJECT: JAVA PROGRAMMING
COURSE CODE: CSA0907

GITHUB LINK: https://github.com/Nithiyasri18/CSA0907-JAVA-192572082/tree/main/ASSIGNMENT

ASSIGNMENT 1

CO1: Java Fundamentals and Object-Oriented Programming

TASK 1 – Understand the Problem and Identify the Objects

---

1. Inputs the system receives and outputs it must produce

Inputs: vehicle registration number, vehicle type (two-wheeler, car, EV; later e-bike and van), entry time, duration of stay in minutes (supplied for testing), pass details (monthly pass or one-day pass) and the kWh used by an EV at a charging slot. The facility layout (number of floors and the number of slots on each floor) is given at start-up.

Outputs: the allocated {floor, slot}, a parking ticket with a unique ticket number and entry time, the parking fee (plus charging fee for an EV) on exit, an error message when the facility is full or an operation is invalid, and a daily collection report.

2. Nouns that become classes and verbs that become methods

Candidate noun	Responsibility (why it is needed)	Verbs → methods
Vehicle (abstract) and subclasses	Holds the registration number and knows its own hourly rate; different types price differently	getRegNo(), calculateFee()
ParkingFacility (slot array)	Owns floors and slots; each slot holds at most one vehicle	allocate(), release()
ParkingTicket	Proof of entry; carries a unique ticket number, slot and entry time	getTicketNo(), getEntry()
ParkingService	Coordinates entry and exit: allocate → ticket → fee → release	enter(), exit()

Candidate noun	Responsibility (why it is needed)	Verbs → methods
Pass (Monthly / One-day)	Decides whether the user parks free	covers(date)
Chargeable (EV capability)	Charging is separate from parking; only EVs have it	chargingFee(kWh)
VehicleFactory	Creates the right Vehicle from the type text, rejects unknown types	create(type, regNo)

3. Business rules every valid operation must satisfy

- A slot cannot hold two vehicles at the same time.
- A ticket number is unique and never reused (it comes from a counter that only increases).
- A vehicle cannot enter twice without leaving (a duplicate registration number is rejected).
- A vehicle cannot exit without an active ticket; a slot that is already free cannot be released.
- Fee = number of started hours × hourly rate, with a daily maximum of 8 hours; a duration cannot be negative.
- A valid monthly pass means free parking; visitors pay the full fee; EV charging is billed separately even for pass holders.
- When no slot is free the vehicle is refused and no ticket is issued.

4. Assumptions about the facility

- The facility has F floors; each floor has its own fixed number of slots, numbered from 0.
- Any vehicle type may use any free slot; a first-free-slot policy is used.
- Entry time is given by the caller (system clock in the live system); for testing the stay is given in minutes.
- Every started hour is billed in full; 0 minutes costs nothing.
- Rates: two-wheeler Rs 10/h, car Rs 30/h, EV Rs 30/h + Rs 12 per kWh.

Alternative Concept: vehicles as Strings in parallel arrays (regNo[], type[], entryTime[])

Three arrays of size n are kept and the same index i describes one vehicle. Finding a vehicle means scanning regNo[] and then reading type[i] and entryTime[i]. Adding a new attribute (for example owner name) needs a new array, and every place that shifts, deletes or sorts vehicles must move that array as well.

```
String[] regNo = new String[n]; String[] type = new String[n]; long[] entry = new
long[n];
int find(String r) {
    for (int i = 0; i < n; i++) if (regNo[i].equals(r)) return i;
    return -1;
}
```

Criterion	Object design (Vehicle[])	Parallel arrays
Search by regNo	O(n) with a plain array (O(1) average with a HashMap)	O(n)
Add a new attribute	Add one field to one class; other code unchanged	Add a new array and update every shift/delete/sort routine
Space per vehicle	One reference (4–8 bytes) + object header (about 12–16 bytes) + fields	Only the data (no object header) – slightly smaller, but the type is stored as a String
Behaviour (fee rules)	Lives inside the object (polymorphism)	Needs if–else on type[i] everywhere
Safety	Fields of one vehicle always stay together	Arrays can get out of step (index bugs)

Conclusion: both searches are O(n) and parallel arrays use a little less memory per vehicle, but the object model is much safer, easier to extend and lets behaviour sit with the data. This is why objects are chosen.

TASK 2 – Choose the Data Representation

1. Data type chosen for each field and why

Field	Type	Reason
Slot number / floor number	int	Small whole numbers used as array indexes; Java arrays need int indexes
Vehicle number	String	Alphanumeric (e.g. TN01AB1111); stored in upper case so lookups are consistent
Entry time	java.time.LocalTime	Avoids manual hour/minute arithmetic; immutable and comparable
Duration	int (minutes)	Whole minutes; int is enough
Fee	double	Rates may have paise and multipliers (×2); BigDecimal is better for real money, double is enough here
Ticket number	int (static counter)	Increases by one for every ticket issued

2. How floors and slots are represented

A jagged two-dimensional array `String[][] slots` is used. `slots[f]` is a separate array for floor f, so each floor can have a different number of slots (for example {20}, {50,50,50} or {2,3,1,2,1}) without wasting memory. A value of `null` means the slot is free; otherwise it holds the registration number of the parked vehicle. A rectangular array would waste memory or force all floors to the same size.

3. Where type conversion / casting is needed, and the risk of precision loss

Billing needs minutes converted to hours. Integer division truncates and gives a wrong fee:

```
int minutes = 130;
double wrong   = minutes / 60 * 30;          // 130/60 = 2 (integer division) ->
60.0  WRONG
double alsoBad = (int)(minutes / 60.0) * 30; // cast truncates 2.17 to 2      ->
60.0  WRONG
int hours      = (minutes + 59) / 60;        // ceiling with integers only    -> 3
double right   = hours * 30.0;               // 90.0
CORRECT
```

The customer stayed 2 h 10 min and every started hour is charged, so the correct fee is Rs 90, but integer division gives Rs 60 – a loss of Rs 30 per vehicle. The design computes the ceiling with `(minutes + 59) / 60` using only integers (exact, no floating-point error) and only then widens `hours` to double (a safe implicit widening) when multiplying by the rate. Narrowing casts such as `(int)` on a double are never used for money.

Execution of Task02/CastingDemo.java (input 130 read with Scanner):

```
$ echo 130 | java CastingDemo
Enter parking duration in minutes: minutes / 60 * rate          = 60.0   (WRONG: integer division)
(int)(minutes / 60.0) * rate  = 60.0   (WRONG: cast truncates)
(minutes + 59) / 60 = 3 h  ->  fee = 90.0   (CORRECT)
Fee lost per vehicle with the wrong method = Rs 30.0

$
```

4. How input is read

A Scanner on System.in is used for interactive input (vehicle type, registration number, minutes). Command-line arguments are not suitable because the program is a long-running, menu-driven system that receives many events, whereas `args[]` is read only once at start-up. Input is validated (non-empty, numeric, non-negative) before use, and `nextLine()` is called after `nextInt()` to consume the leftover newline (as in CastingDemo).

Alternative Concept: 1-D array with index = floor × slotsPerFloor + slot, or ArrayList<String[]>

1-D array: works cleanly only when every floor has the same number of slots S. With different sizes the formula `floor × S + slot` breaks, so a prefix-sum array `start[f]` (index of the first slot of floor f) must be kept: `index = start[f] + slot`. Locating a slot is still O(1). Space is exactly the total number of slots T = Σ slots(f), plus F integers for start[].

ArrayList<String[]>: each element is the String[] of one floor, so different sizes are natural and floors can be added at run time. Locating a slot is O(1) (`get(f)[s]`) with a small overhead for the list; space is T references + F array headers + the list.

	Jagged String[][]	**1-D array + start[]**	**ArrayList<String[]>**
Locate slot	O(1)	O(1)	O(1)
Space	T + F headers	T + F ints	T + F headers + list
Different floor sizes	Natural	Needs start[]	Natural

	Jagged String[][][]	1-D array + start[]	ArrayList<String[]>
Add a floor later	Re-create array	Re-create array	list.add() – easy

TASK 3 – Design the Control Logic

1. How the program searches for a free slot, and when it stops

Nested for-loops: the outer loop walks the floors from 0, the inner loop walks the slots of that floor from 0. The first slot whose value is `null` is taken, the vehicle number is stored there and the method returns immediately with {floor, slot}. The search stops at the first free slot (best case O(1)) or after every slot has been checked (worst case O(F × S)).

2. How fee slabs are expressed

Slab rule used: every started hour is charged at the vehicle's hourly rate (so the first hour is a minimum charge), there is a daily maximum of 8 hours' worth of fee, and 0 minutes is free. These are expressed with if–else ranges (and `Math.min` for the cap), while the vehicle type → rate choice is done by polymorphism (or a switch in the factory).

3. Use of break, continue and return

- `return` inside the inner loop ends both loops at once as soon as a slot is allocated (no labelled break needed).
- `continue` skips an occupied slot (`slots[f][s] != null`) without running the rest of the loop body.
- `break` leaves a `switch` case; a labelled `break outer;` is the alternative to `return` when the result is kept in a variable.
- Early `return false` in release() rejects an invalid floor/slot before any array access, preventing ArrayIndexOutOfBoundsException.

4. What happens when no suitable slot is free

allocate() returns `null`, the service refuses the entry, no ticket is issued (so the ticket counter is not consumed) and the message “Facility full” is shown (test 12 and 13 in Task 11).

Pseudocode

```
ALGORITHM AllocateSlot(vehicleNo)
INPUT  : vehicleNo
```

```
OUTPUT : {floor, slot} or NULL
BEGIN
    FOR floor <- 0 TO numberOfFloors - 1 DO
        FOR slot <- 0 TO slotsOnFloor[floor] - 1 DO
            IF slots[floor][slot] != NULL THEN
                CONTINUE
            END IF
            slots[floor][slot] <- vehicleNo
            RETURN {floor, slot}
        END FOR
    END FOR
    RETURN NULL
END


ALGORITHM ComputeFee(minutes, rate)
INPUT  : minutes (>= 0), rate per hour
OUTPUT : fee
BEGIN
    IF minutes < 0 THEN RAISE error END IF
    IF minutes = 0 THEN RETURN 0 END IF
    hours <- (minutes + 59) DIV 60          // ceiling, integers only
    IF hours > 8 THEN hours <- 8 END IF     // daily maximum
    fee <- hours * rate
    RETURN fee
END
```

Note: the pseudocode follows the usual structured conventions (ALGORITHM / INPUT / OUTPUT, BEGIN / END, ← for assignment, END IF / END FOR). Keywords should be adjusted if the SIMATS standard sheet uses different ones.

Complete the Code – ParkingFacility (completed code)

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number, or null if the slot is free
    private final String[][] slots;


    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }


    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {                // TODO 1
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
```

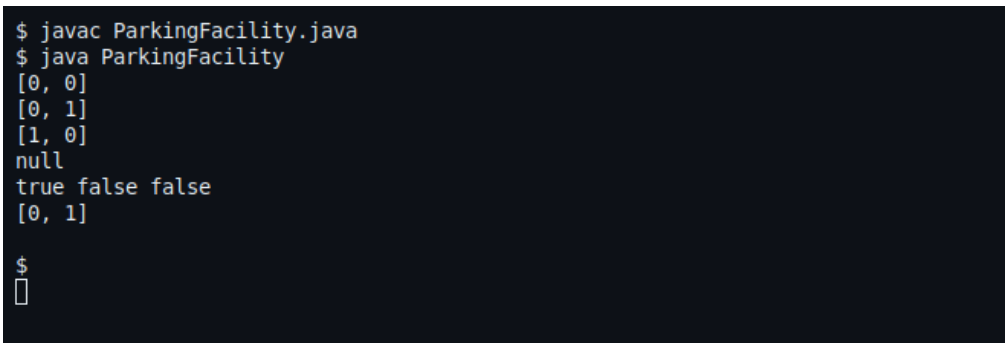
```
        }
        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;        // TODO 2
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Output (expected and obtained):

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```



Alternative Concept: free-slot count per floor and a stack of free slot numbers

Keep `int[] freeCount` (one counter per floor) and, for each floor, a stack of free slot numbers (initially all slots pushed). allocate(): find a floor with freeCount[f] > 0, pop a slot number from its stack, decrement freeCount[f] and mark the slot. release(): validate

the position, push the slot number back and increment freeCount[f]. A global free count gives “facility full” in O(1).

```
int[] allocate(String v) {
    for (int f = 0; f < F; f++)
        if (freeCount[f] > 0) {
            int s = freeStack[f].pop(); freeCount[f]--; slots[f][s] = v;
            return new int[]{f, s};
        }
    return null;
}
boolean release(int f, int s) {
    /* validate */ slots[f][s] = null; freeStack[f].push(s); freeCount[f]++; return true;
}
```

Approach	allocate()	release()	Extra space
Scan (used)	O(F × S) worst case, O(1) best	O(1)	O(1)
Counter + stack	O(F) (O(1) with a global “non-full floor” pointer)	O(1)	O(F) counters + O(F × S) stack entries

The stack approach is faster for large facilities (5 × 100 slots) at the cost of extra memory and more code to keep the stack and the slot array consistent. It also allocates the most recently released slot, not the lowest-numbered one.

TASK 4 – Design Classes, Objects and Constructors

1. Fields and methods of each class

Class	Fields	Methods
Vehicle (abstract)	private final String regNo	getRegNo(), ratePerHour() [abstract], calculateFee() (overloaded), billableHours(), equals(), hashCode(), toString()
TwoWheeler / Car / ElectricCar	(inherit regNo)	ratePerHour(); ElectricCar adds chargingFee()
ParkingTicket	static counter; ticketNo, vehicle, floor, slot, entry	getters; copy constructor
ParkingFacility	String[][] slots	allocate(), release(), totalSlots()
ParkingService	facility, Map active, totalCollection	enter(), exit(), getTotalCollection()

2. Default, parameterised and overloaded constructors and chaining

```
class ParkingTicket {
    private static int counter = 0;
    private final int ticketNo;
    private final Vehicle vehicle;
    private final int floor, slot;
    private final LocalTime entry;


    public ParkingTicket(Vehicle v, int floor, int slot, LocalTime entry) {
        if (v == null || entry == null) throw new
IllegalArgumentException("Vehicle and entry time required");
        this.ticketNo = ++counter;
        this.vehicle = v; this.floor = floor; this.slot = slot; this.entry =
entry;
    }
    public ParkingTicket(Vehicle v, int floor, int slot) {      // overloaded:
entry = now
        this(v, floor, slot, LocalTime.now());
    }
    public ParkingTicket(ParkingTicket t) {                     // copy
constructor (duplicate ticket)
        this.ticketNo = t.ticketNo; this.vehicle = t.vehicle;
        this.floor = t.floor; this.slot = t.slot; this.entry = t.entry;
    }
    public int getTicketNo() { return ticketNo; }
    ...
```

- **Default constructor:** not provided for ParkingTicket, because a ticket without a vehicle or slot would be invalid. ParkingFacility has one that builds the standard 3 × 50 layout and calls `this(new int[]{50, 50, 50})`.
- **Parameterised constructor:** used when all data is known (normal entry); it validates and takes the next number from the counter.
- **Overloaded constructor:** lets the caller omit the entry time; it chains with `this(...)` so the validation and counter logic exist in only one place.
- **this keyword:** separates field and parameter names (`this.floor = floor`) and calls another constructor.

3. Is a copy constructor needed?

Yes, for re-issuing a duplicate ticket when the customer loses the paper ticket. The copy keeps the same ticket number (it is the same parking record), so it does not increment the counter. Because the fields are final and Vehicle is immutable, a shallow copy is safe.

4. How objects are stored in memory

Reference variables (e.g. `ParkingTicket t`) live on the stack inside the method frame; the objects themselves (`new ParkingTicket(...)`, the Vehicle, the String[][] arrays) live on the heap. The variable holds only the address, so `t2 = t1` copies the reference and both point to the same object, whereas the copy constructor creates a second, separate heap object. `static` fields (counter) live in the class's memory area, not in any object.

Execution of Task04/TicketDemo.java:

```
$ javac TicketDemo.java
$ java TicketDemo
t1 ticketNo        = 1, entry = 08:00
t2 ticketNo        = 2, entry set to now = true
dup ticketNo       = 1 (same record as t1)
dup == t1          = false   (different object on the heap)
alias == t1        = true   (same object, two references)
Tickets issued     = 2   (copy did not use the counter)

$
```

Alternative Concept: default constructor + setters, or static factory methods

Default constructor + setters: `new ParkingTicket(); t.setVehicle(v); t.setSlot(...)`. The object exists in an invalid, half-built state between calls, fields cannot be final, and a forgotten setter leaves null values. **Static factory:** `ParkingTicket.issue(v, floor, slot)` has a descriptive name, can validate before creating and can return a cached object or a subclass, but the class still needs a (private) constructor.

	Overloaded constructors	**Setters**	**Static factory**
Guarantees valid object	Yes – all required data at creation	No – can be incomplete	Yes – validates before new
Immutability (final)	Possible	Not possible	Possible
Time for n tickets	O(n)	O(n) (k setter calls each)	O(n)
Space for n tickets	O(n)	O(n)	O(n) (O(1) extra if objects are reused)

TASK 5 – Apply static Members and the Object Life Cycle

1. Where static variables are needed

The ticket counter (`private static int counter`) belongs to the class as a whole: one counter must produce unique numbers for every ticket ever created. An instance variable would start at 0 for each new object, so every ticket would get number 1 and the

rule “a ticket number is never reused” would break. A static `totalOccupancy` counter (incremented on entry, decremented on exit) is another example.

2. Static methods and static blocks

Static methods are used for behaviour that needs no object: `VehicleFactory.create(type, regNo)`, `Chargeable.isValidUnits(kWh)` and a utility such as `Fees.hoursFor(minutes)`. A static block runs once when the class is loaded and is suitable for filling a rate table, e.g. `static { RATES.put("bike", 10.0); RATES.put("car", 30.0); }`.

3. Life of a ParkingTicket after the vehicle leaves

On exit the ticket is removed from the active-tickets map (`active.remove(regNo)`) and the slot is released. When no reference to the ticket remains (local variable out of scope, removed from the map) it is unreachable and eligible for garbage collection; the JVM frees it at some later, unpredictable time. If a history list kept a reference it would stay alive. The static counter is not reset, so the number is never reused.

4. How objects are passed and returned

Java is always pass-by-value; for objects the value passed is the reference. enter() creates a Vehicle and a ParkingTicket and returns the ticket reference; exit() takes primitives, a String and a Pass reference. A method that changes a field of a passed object affects the caller's object, but re-assigning the parameter to a new object does not. allocate() returns a new int[] (an object) holding {floor, slot}.

Execution of Task05/StaticDemo.java:

```
$ javac StaticDemo.java
$ java StaticDemo
STATIC counter   : north=1, south=2, north=3, south=4
INSTANCE counter : north=1, south=1, north=2, south=2   <- numbers repeat
[static block executed once] rates loaded: {car=30.0, bike=10.0}
Fees.hoursFor(130) = 3
Rate of car        = 30.0
After changeField(t): t.floor = 3  (object changed)
After reassign(t)   : t.floor = 3  (caller's reference unchanged)
Ticket reference set to null: the object is now eligible for GC

$
```

Alternative Concept: counter as an instance variable of ParkingFacility

With two facilities (north and south gates) each would have its own counter and both would issue 1, 2, 3 … – the INSTANCE line in the screenshot shows the numbers

repeating – so a ticket number could not identify one parking event. Solutions: (a) one static counter shared by both (unique across the campus, but the number does not show which facility); (b) instance counters with the ticket id made of (facilityId, number), e.g. N-001 / S-001; (c) a separate TicketIdGenerator singleton given to both.

Design	**Space for k facilities and n tickets**	**Uniqueness**
Static counter	O(1) – one int	Unique across all facilities
Instance counter	O(k) – one int per facility	Unique only within a facility (needs facilityId)

In both designs the tickets themselves take O(n) space; the counter's own cost is negligible, so the choice is made on correctness, not memory.

TASK 6 – Design the Inheritance Hierarchy

1. Superclass, subclasses and type of inheritance

Vehicle is the abstract superclass. TwoWheeler and Car extend Vehicle, and ElectricCar extends Car. This combines hierarchical inheritance (TwoWheeler and Car both extend Vehicle) and multilevel inheritance (Vehicle → Car → ElectricCar). Java has no multiple inheritance of classes; the EV's extra capability comes from the Chargeable interface. Users (student, staff, visitor) are modelled with the Pass concept instead of subclasses, because they differ only in data (the pass), not in behaviour.

2. What is inherited and what is added

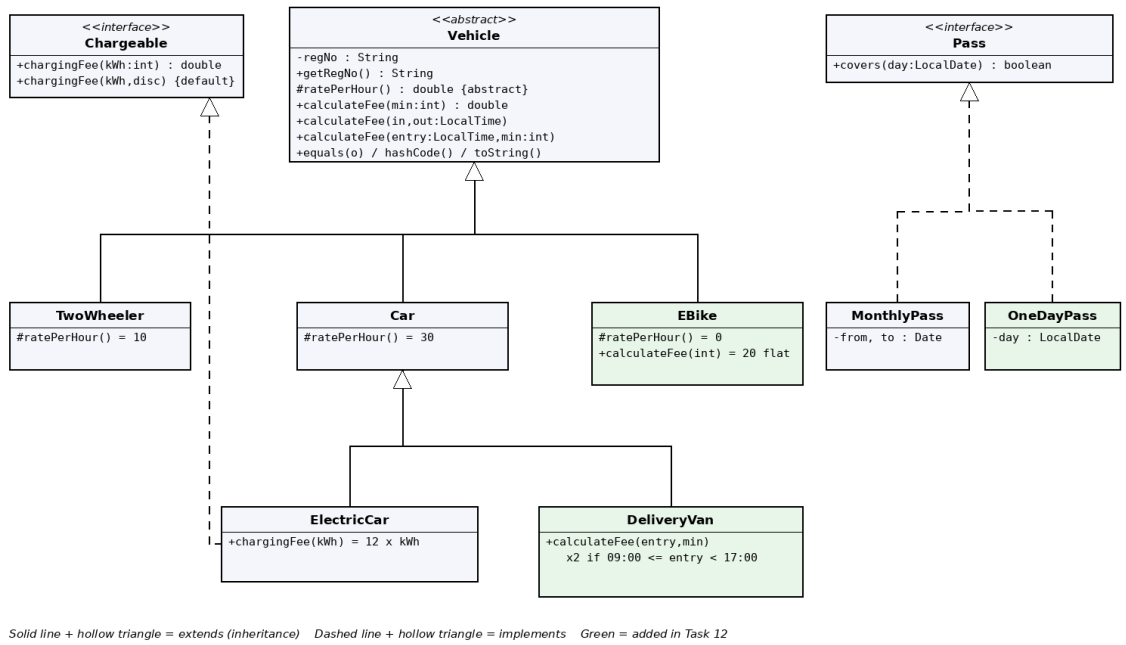
Class	**Inherited**	**Added / changed**
Vehicle	–	regNo, getRegNo(), calculateFee(), equals(), hashCode(); abstract ratePerHour()
TwoWheeler	All of Vehicle	ratePerHour() = 10
Car	All of Vehicle	ratePerHour() = 30

Class	Inherited	Added / changed
ElectricCar	All of Car (including rate 30)	chargingFee(kWh) = 12 × kWh (from Chargeable)

3. Use of super

`super(regNo)` in every subclass constructor passes the registration number up to Vehicle, which validates and stores the private field. A subclass can call an overridden superclass method with `super.method()`. `regNo` is private, so subclasses use `getRegNo()` and not direct access.

4. UML class diagram



5. Constructor execution order for new ElectricCar("TN03C3")

```
1. new ElectricCar("TN03C3") -> ElectricCar(String) starts, first statement is super(regNo)
2. Car(String)      -> starts, first statement is super(regNo)
3. Vehicle(String)  -> (Object() runs first) then this.regNo = "TN03C3"  [body runs]
4. Car body completes
5. ElectricCar body completes
```

Constructors are entered from the subclass, but their bodies complete from the top: Object → Vehicle → Car → ElectricCar. The trace below is printed by the real constructors (Task06/ConstructorOrder.java):

```
$ javac ConstructorOrder.java
$ java ConstructorOrder
1. ElectricCar(String) starts, first statement is super(regNo)
2. Car(String) starts, first statement is super(regNo)
3. Vehicle(String) body runs: this.regNo = TN03C3   (Object() ran just before)
4. Car(String) body completes
5. ElectricCar(String) body completes
Object created: ElectricCar TN03C3

$
```

Alternative Concept: composition (Vehicle has-a VehicleType holding the rate)

```
class VehicleType { final String name; final double rate; ... }
class Vehicle { private final String regNo; private final VehicleType type;
    double calculateFee(int min) { return ((min + 59) / 60) * type.rate; } }
Vehicle v = new Vehicle("TN02B2", new VehicleType("CAR", 30));
```

Criterion	Inheritance	Composition
Add a new vehicle type	New subclass (a new class file)	New VehicleType object (data only, no new class)
Fee computation time	O(1) – virtual call	O(1) – field read
Space per vehicle	Object header + regNo	Object header + regNo + one reference to a shared VehicleType
Different behaviour (EV charging)	Natural with overriding	Needs a strategy object or extra if checks

Composition is better when types differ only in data (a rate); inheritance is better when types differ in behaviour. Because the e-bike (flat fee) and van (time-based fee) need different behaviour, inheritance with an overridable pricing method remains justified.

TASK 7 – Override and Specialise Behaviour

1. Which methods are overridden and why

`ratePerHour()` is abstract in Vehicle and overridden in TwoWheeler (10) and Car (30), so each type supplies its own rate. `calculateFee()` is written once in Vehicle using `ratePerHour()` (template-method style); subclasses such as EBike (flat fee) and DeliveryVan (peak-hour multiplier) override it when their rule differs. `toString()`, `equals()` and `hashCode()` are overridden from Object. `chargingFee()` implements the Chargeable interface in ElectricCar.

2. @Override and the rules of overriding

- `@Override` makes the compiler check that a method really overrides one, so a typo such as `ratePerhour()` becomes a compile error instead of a silent new method.

- The signature (name and parameter types) must be identical; the return type must be the same or a subtype (covariant); access may stay the same or become wider (protected → public is allowed, not the reverse); no broader checked exceptions may be thrown.
- `static`, `private` and `final` methods cannot be overridden.

3. Use of final

- **final field:** `private final String regNo` – a registration number never changes after creation; `ticketNo` is also final.
- **final method:** `billableHours()` is `public final` in Vehicle so no subclass can break the rule "a started hour is charged in full" or skip the negative-duration check.
- **final class:** a utility or ticket class can be final so it cannot be extended; String is an example.

4. toString(), equals() and hashCode()

```
@Override public boolean equals(Object o) {
    return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
}
@Override public int hashCode() { return regNo.hashCode(); }
@Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
```

Two Vehicle objects with the same registration number are equal. `hashCode()` is overridden at the same time because equal objects must have equal hash codes – otherwise a HashMap or HashSet would not find the vehicle. Both use only `regNo`.

Alternative Concept: one calculateFee() with a switch on a type field

```
double calculateFee(int minutes) {
    int hours = (minutes + 59) / 60;
    switch (type) {
        case "BIKE": return hours * 10;
        case "CAR":  return hours * 30;
        case "EV":   return hours * 30;
        default: throw new IllegalArgumentException();
    }
}
```

Criterion	Overriding	switch on type
Time per fee calculation	O(1) – dynamic dispatch	O(1) for a switch on a String/enum, O(k) as an if–else chain over k types
Classes changed when a type is added	0 existing (add 1 new subclass)	1 (the class holding the switch is edited and retested) plus every other switch on type
Open/Closed principle	Followed	Violated

TASK 8 – Apply Polymorphism

1. Compile-time polymorphism (overloading)

```
public double calculateFee(int minutes) { ... }
public double calculateFee(LocalTime in, LocalTime out) {
    return calculateFee((int) Duration.between(in, out).toMinutes());
}
public double calculateFee(LocalTime entry, int minutes) { return calculateFee(minutes); } //
Task 12
```

The compiler chooses the method from the number and types of the arguments: `calculateFee(130)` matches (int); `calculateFee(in, out)` matches (LocalTime, LocalTime); `calculateFee(entry, 130)` matches (LocalTime, int). The return type alone cannot separate overloads.

2. One array of Vehicle references holding different types (upcasting)

`Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"), new ElectricCar("TN03C3") };` Each object is implicitly upcast to Vehicle, so one loop can process all vehicles. Only methods declared in Vehicle can be called through the reference; `chargingFee()` needs an `instanceof Chargeable` test.

3. Dynamic method dispatch

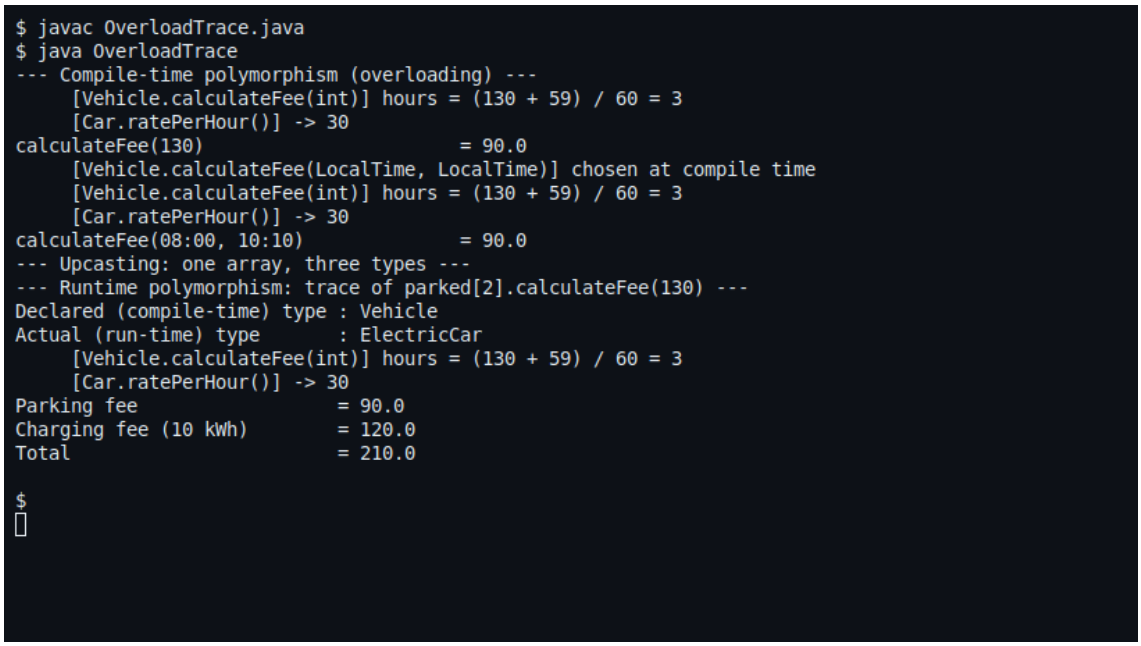
At run time the JVM looks at the actual class of the object (not the declared type of the reference) and runs that class's version of the overridden method.

4. Why this avoids long if–else chains

The loop `for (Vehicle v : parked) v.calculateFee(130)` has no type tests. When a new vehicle type is added, the loop does not change.

5. Trace of one call

Item	Value for v.calculateFee(130) (v = parked[2])
Declared (compile-time) type	Vehicle
Actual (run-time) type	ElectricCar
Method executed	calculateFee(int) is found in Vehicle (inherited by Car and ElectricCar, not overridden)
Inside it	hours = (130 + 59) / 60 = 3; calls this.ratePerHour(), which is dispatched to Car.ratePerHour() = 30
Result	3 × 30 = 90.0; the program then adds chargingFee(10) = 120, total Rs 210.0



Output of Task08/OverloadTrace.java – the printed lines come from the methods that actually ran

Complete the Code – Vehicle hierarchy (completed code)

```
interface Chargeable {
    double chargingFee(int kWh);
}


abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();


    /** Every started hour is charged in full. */
    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;          // TODO 1: ceiling with integers only
        return hours * ratePerHour();
    }
    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}


class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}

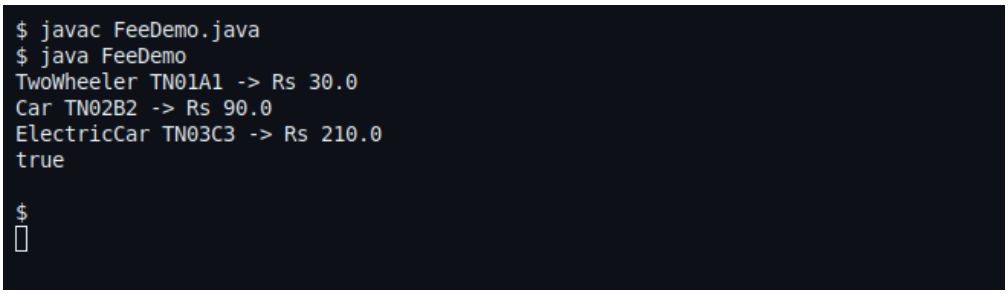

class Car extends Vehicle {                        // TODO 2
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}
```

```
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) { return kWh * 12; }   // TODO 3
}


public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
                new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);       // dynamic method dispatch
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
                    + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Output (expected and obtained):

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```



Why does the last line print true? Is this correct for the system?

`equals()` in Vehicle compares only the registration number and tests `instanceof Vehicle` (not `getClass()`), so a Car and an ElectricCar with the same regNo "TN02B2" are equal. For this system this is the correct behaviour: a registration number identifies one physical vehicle, so two records with the same number must be treated as the same vehicle – exactly what duplicate-entry detection needs. The vehicle type is not part of its identity. It would be wrong only if the system had to tell apart different types with the same number, which cannot happen in reality.

Alternative Concept: instanceof checks and casting for each vehicle type

```
double fee;
if (v instanceof ElectricCar) fee = hours(m) * 30 + ((ElectricCar) v).chargingFee(kWh);
else if (v instanceof Car)    fee = hours(m) * 30;   // subclass must be tested before
superclass!
else if (v instanceof TwoWheeler) fee = hours(m) * 10;
```

Time per call is O(1) for dynamic dispatch (a virtual-table lookup) but up to O(k) type checks for k types, and the order of the checks matters (ElectricCar before Car). Each new type means editing every such chain in the program, the code grows with every type, and a forgotten branch still compiles but fails at run time; dispatch needs only a new class.

TASK 9 – Introduce Abstraction

1. Which class and which methods are abstract

Vehicle is abstract because "a vehicle" is a general idea; only specific types (car, bike …) are ever parked, so it must not be instantiated. `ratePerHour()` is abstract because every concrete type must state its own rate and the compiler forces this. `calculateFee()` is concrete because the "started hour" rule is the same for all.

2. Which capability is an interface

`Chargeable` (chargingFee) is an interface because charging is a capability that does not belong to every vehicle and cuts across the hierarchy: an ElectricCar (and later an e-bike) can be Chargeable. `Pass` (covers(date)) is another interface, implemented by MonthlyPass and OneDayPass; a `Payable` interface for fee-paying users would be a further example.

3. Why an abstract class in one place and an interface in another

- **Abstract class:** shares state (regNo), a constructor and common code (calculateFee) among closely related classes – an "is-a" relationship – and a class can extend only one.
- **Interface:** defines only a contract ("can do") with no state; unrelated classes can implement it, and one class can implement many.

4. Multiple behaviours through interfaces

`class ElectricCar extends Car implements Chargeable, Payable` – Java allows a class to implement any number of interfaces, which is how it gets several behaviours despite having a single superclass.

5. Default or static method in an interface

```
interface Chargeable {
    double chargingFee(int kWh);
    default double chargingFee(int kWh, double discountPercent) {   // default method
        return chargingFee(kWh) * (1 - discountPercent / 100);
    }
    static boolean isValidUnits(int kWh) { return kWh >= 0; }       // static method
}
```

A default method lets an interface gain a new method without breaking the classes that already implement it; a static method keeps a related helper with the interface. Both are used in the project (ElectricCar calls `Chargeable.isValidUnits(kWh)`).

Alternative Concept: Vehicle as an interface with default methods, Chargeable as an abstract class

As an interface, Vehicle can no longer hold the `regNo` field (interface fields are public static final constants), cannot have a constructor and cannot define `equals()/hashCode()` as default methods (default methods cannot override Object's methods). Each class would need its own regNo and constructor with duplicated code. Chargeable as an abstract class would force an EV to extend it; with single inheritance an ElectricCar could then no longer extend Car, and a class could not have two such capabilities. In both cases the call time stays O(1) (interface dispatch is only slightly slower than class dispatch because of the itable lookup) and the space per object is the same or slightly larger (duplicated fields); the real difference is in what can be expressed, not in speed.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers

1. Package structure

```
parking.model   : Vehicle, TwoWheeler, Car, ElectricCar, EBike, DeliveryVan, ParkingTicket,
                  Chargeable, Pass, MonthlyPass, OneDayPass
parking.service : ParkingFacility, ParkingService, VehicleFactory
parking.app     : Main (test driver; a Scanner-based menu can be added here)
```

Packages group classes by responsibility (data, logic, user interface): the interface depends on the service and the service on the model, never the other way. The packaged project (Task10/src) compiles and runs:

```
$ find src -name "*.java" | sort
src/parking/app/Main.java
src/parking/model/Car.java
src/parking/model/Chargeable.java
src/parking/model/DeliveryVan.java
src/parking/model/EBike.java
src/parking/model/ElectricCar.java
src/parking/model/MonthlyPass.java
src/parking/model/OneDayPass.java
src/parking/model/ParkingTicket.java
src/parking/model/Pass.java
src/parking/model/TwoWheeler.java
src/parking/model/Vehicle.java
src/parking/service/ParkingFacility.java
src/parking/service/ParkingService.java
src/parking/service/VehicleFactory.java
$ javac -d out $(find src -name "*.java")
$ java -cp out parking.app.Main | tail -n 4

New-category tests: 11 passed, 0 failed

TOTAL: 32 passed, 0 failed

$
```

2. Access modifier for each member

Member	Modifier	Why
regNo, ticketNo, slots, counter	private	State may change only through validated methods
getRegNo(), calculateFee(), allocate()	public	Part of the public API
Vehicle constructor, ratePerHour()	protected	Used by subclasses (any package), not by clients
Helper class used inside one package	default (no modifier)	Visible only inside its own package
Subclass constructors (Car, …)	public	Needed across packages by VehicleFactory

3. Validating getters and setters

```
public void setMinutes(int minutes) {
    if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
    this.minutes = minutes;
}
public boolean release(int floor, int slot) {     // slot-number validation
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    ...
}
```

Every write goes through one method that rejects invalid data (negative duration, empty registration number, invalid floor/slot), so an object can never be put in an invalid state.

4. Wrapper classes and autoboxing

Collections cannot store primitives, so `Map<String, Integer>` uses the wrapper Integer and autoboxing converts int ↔ Integer automatically. Risks: (a) == on Integer objects compares references – values outside −128…127 are separate objects, so `Integer a = 1000, b = 1000; a == b` is false (use `equals`); (b) unboxing a null Integer throws NullPointerException (e.g. `map.get("x")` for a missing key); (c) boxing in long loops creates many objects and is slower than primitives.

```
$ javac WrapperDemo.java
$ java WrapperDemo
setMinutes(130) accepted -> 130
setMinutes(-5) rejected  -> Duration cannot be negative
autoboxing/unboxing works -> 3
Integer 1000 == 1000      -> false   (different objects)
Integer 1000 .equals 1000 -> true
Integer  100 == 100       -> true   (cached -128..127)
unboxing a missing key    -> NullPointerException

$
```

Alternative Concept: public fields with validation done by the calling code

`public int minutes;` and each caller writes `if (m >= 0) v.minutes = m;`. What can go wrong: any caller can forget the check or write a different one (e.g. `> 0` instead of `>= 0`), a negative duration then silently gives a negative fee, the class can never change its internal representation without breaking every caller, and no invariant can be guaranteed.

	Private + validating setter	**Public field + caller checks**
Time cost per update	O(1) – one comparison	O(1) – one comparison
Places where the check is written	1	One per call site (c places, e.g. 5–20)
Risk	Low	Check forgotten or inconsistent

TASK 11 – Implement, Test and Defend the Design

1. Complete implementation

The complete program was written in one file for execution (Task11/Main.java, Java 21) and is also split into the packages of Task 10 (Task10/src). The key class is ParkingService:

```
class ParkingService {
    private final ParkingFacility facility;
    private final Map<String, ParkingTicket> active = new HashMap<>();
    private double totalCollection = 0;


    public ParkingService(ParkingFacility facility) { this.facility = facility; }


    public ParkingTicket enter(String type, String regNo, LocalTime time) {
        Vehicle v = VehicleFactory.create(type, regNo);              // throws on unknown
type
        if (active.containsKey(v.getRegNo()))
            throw new IllegalStateException("Already parked: " + v.getRegNo());
        int[] pos = facility.allocate(v.getRegNo());
        if (pos == null) throw new IllegalStateException("Facility full");   // no ticket
issued
        ParkingTicket t = new ParkingTicket(v, pos[0], pos[1], time);
        active.put(v.getRegNo(), t);
        return t;
    }


    public double exit(String regNo, int minutes, int kWh, Pass pass, LocalDate today) {
        String key = regNo == null ? "" : regNo.trim().toUpperCase();
        ParkingTicket t = active.get(key);
        if (t == null) throw new IllegalStateException("No active ticket");
        Vehicle v = t.getVehicle();
        v.billableHours(minutes);                                    // validates before any
change
        double fee = (pass != null && pass.covers(today)) ? 0 : v.calculateFee(minutes);
        if (v instanceof Chargeable c) fee += c.chargingFee(kWh);     // charging billed
separately
        active.remove(key);
        facility.release(t.getFloor(), t.getSlot());
        totalCollection += fee;
        return fee;
    }


    public double getTotalCollection() { return totalCollection; }
    public int activeCount() { return active.size(); }
}
```

The remaining classes (Vehicle hierarchy, ParkingTicket, ParkingFacility, VehicleFactory, Pass classes) are in the source files and are exactly as described in Tasks 4, 6, 7, 8 and 12.

2. Test table (facility layout {2,3,1,2,1} = 9 slots unless stated)

The table below is the program's own output (copied from Task11/output.txt); “Expected” is fixed in the code before the run, so a wrong result would print FAIL.

No.	Test case	Expected	Actual	Result
1	Bike, car, EV, car enter	T1[0,0];T2[0,1];T3[1,0];T4[1,1]	T1[0,0];T2[0,1];T3[1,0];T4[1,1]	Pass
2	Bike, 130 min	30.0	30.0	Pass
3	Car, 130 min	90.0	90.0	Pass
4	EV, 130 min, 10 kWh	210.0	210.0	Pass
5	Car with valid monthly pass	0.0	0.0	Pass
6	Daily total (hand = 330)	330.0	330.0	Pass
7	Slot reused after exit	[0,0]	[0,0]	Pass
8	Boundary: 0 minutes	0.0	0.0	Pass
9	Boundary: exactly 60 minutes	30.0	30.0	Pass
10	Boundary: 61 minutes	60.0	60.0	Pass
11	Daily maximum: 600 minutes	240.0	240.0	Pass
12	Last free slot, then another	[0,0] then ISE: Facility full	[0,0] then ISE: Facility full	Pass
13	Full: ticket counter unchanged	true	true	Pass
14	Duplicate registration	ISE: Already parked: TN09J9	ISE: Already parked: TN09J9	Pass
15	Unknown type (tractor)	IAE: Unknown vehicle type: tractor	IAE: Unknown vehicle type: tractor	Pass
16	Invalid slot release(9, 9)	false	false	Pass
17	Release a slot that is free	false	false	Pass
18	Empty registration number	IAE: Registration number required	IAE: Registration number required	Pass
19	Negative duration	IAE: Duration cannot be negative	IAE: Duration cannot be negative	Pass
20	Exit without a ticket	ISE: No active ticket	ISE: No active ticket	Pass
21	Car equals EV, same regNo	true	true	Pass

```
$ javac Main.java Benchmark.java
$ java Main | head -n 20
CAMPUS SMART PARKING SYSTEM
---------------------------
ENTRY (all at 08:00)
  Ticket 1 | TwoWheeler  TN01AB1111 | floor 0 slot 0 | entry 08:00
  Ticket 2 | Car         TN02CD2222 | floor 0 slot 1 | entry 08:00
  Ticket 3 | ElectricCar TN03EF3333 | floor 1 slot 0 | entry 08:00
  Ticket 4 | Car         TN04GH4444 | floor 1 slot 1 | entry 08:00

EXIT (each stayed 130 minutes -> 3 billable hours)
  Bike Fee: Rs 30.0
  Car Fee: Rs 90.0
  EV Fee: Rs 210.0  (90.0 parking + 120.0 charging)
  Monthly Pass Car Fee: Rs 0.0

DAILY COLLECTION REPORT
  Total Collection: Rs 330.0   (hand calculation: 30+90+210+0 = 330.0)

TEST TABLE
No.  Test case                      Expected                          Actual                            Result
-------------------------------------------------------------------------------------------------------------------

$
```

```
$ java Main | tail -n +21
1    Bike, car, EV, car enter         T1[0,0];T2[0,1];T3[1,0];T4[1,1]     T1[0,0];T2[0,1];T3[1,0];T4[1,1]     Pass
2    Bike, 130 min                    30.0                                30.0                                Pass
3    Car, 130 min                     90.0                                90.0                                Pass
4    EV, 130 min, 10 kWh              210.0                               210.0                               Pass
5    Car with valid monthly pass      0.0                                 0.0                                 Pass
6    Daily total (hand = 330)         330.0                               330.0                               Pass
7    Slot reused after exit           [0,0]                               [0,0]                               Pass
8    Boundary: 0 minutes              0.0                                 0.0                                 Pass
9    Boundary: exactly 60 minutes     30.0                                30.0                                Pass
10   Boundary: 61 minutes             60.0                                60.0                                Pass
11   Daily maximum: 600 minutes       240.0                               240.0                               Pass
12   Last free slot, then another     [0,0] then ISE: Facility full       [0,0] then ISE: Facility full       Pass
13   Full: ticket counter unchanged   true                                true                                Pass
14   Duplicate registration           ISE: Already parked: TN09J9         ISE: Already parked: TN09J9         Pass
15   Unknown type (tractor)           IAE: Unknown vehicle type: tractor  IAE: Unknown vehicle type: tractor  Pass
16   Invalid slot release(9, 9)       false                               false                               Pass
17   Release a slot that is free      false                               false                               Pass
18   Empty registration number        IAE: Registration number required   IAE: Registration number required   Pass
19   Negative duration                IAE: Duration cannot be negative    IAE: Duration cannot be negative    Pass
20   Exit without a ticket            ISE: No active ticket               ISE: No active ticket               Pass
21   Car equals EV, same regNo        true                                true                                Pass

Original-design tests: 21 passed, 0 failed

$
```

3. Hand-calculated day scenario

Four vehicles enter at 8:00 and each stays 130 minutes (hours billed = ⌈130/60⌉ = 3): bike 3 × 10 = Rs 30; car 3 × 30 = Rs 90; EV 3 × 30 = 90 plus 10 kWh × 12 = 120, so Rs 210; car with a valid monthly pass = Rs 0. Total by hand = 30 + 90 + 210 + 0 = **Rs 330**. The program printed “Total Collection: Rs 330.0” – they match (test 6).

4. How well the design follows the OOP principles, and one limitation

- **Encapsulation:** all fields are private; the registration number is validated in the constructor and cannot be changed (final); invalid slot, duration and registration inputs are rejected (tests 16–20).
- **Inheritance:** the common billing rule is written once in Vehicle; TwoWheeler, Car and ElectricCar reuse it.
- **Polymorphism:** calculateFee() is selected at run time; ParkingService has no vehicle-type tests (only the Chargeable check).

- **Abstraction:** Vehicle (abstract), Chargeable and Pass (interfaces) separate "what" from "how".
- **Limitation:** allocate() scans linearly (O(F × S)) and the program is not thread-safe, so two gates calling it at the same time could assign the same slot. Free-slot stacks and synchronized methods would fix this.

Alternative Concept: active tickets in ArrayList<ParkingTicket> or HashMap<String, ParkingTicket>

ArrayList: on exit, scan the list comparing `getVehicle().getRegNo()` until a match is found – O(n); removing from the middle shifts later elements (another O(n)). **HashMap** keyed by registration number (used in this solution): get/remove are O(1) on average and the duplicate-entry check (`containsKey`) is O(1) too.

```
// ArrayList version of exit()
for (int i = 0; i < list.size(); i++)
    if (list.get(i).getVehicle().getRegNo().equals(reg)) { t = list.remove(i); break; }  //
O(n)
// HashMap version
t = active.remove(reg);                                                                  //
O(1) average
```

Measurement (Task11/Benchmark.java, search for the vehicle that entered last; median of 101 timed runs after 300 warm-up runs; Java 21):

n parked	ArrayList find (ns)	HashMap find (ns)	ArrayList memory (KB)	HashMap memory (KB)
1000	3467	125	3	39
10000	33333	133	39	337
100000	473378	629	390	3758

```
$ java Benchmark
n parked   ArrayList (ns)   HashMap (ns)     ArrayList (KB)     HashMap (KB)
-------------------------------------------------------------------------------
1000       3467            125              3                  39
10000      33333           133              39                 337
100000     473378          629              390                3758
(memory = container only; the ticket objects themselves are shared and counted once)

$
```

The list time grows roughly in proportion to n (about 3.5 µs → 33 µs → 470 µs for 1,000 → 10,000 → 100,000), i.e. O(n), while the HashMap time stays in the sub-microsecond range, i.e. O(1) average. Repeated runs gave the same trend (list ≈ 3.4–3.5 µs, 33–56 µs and 357–473 µs; map between 72 ns and 629 ns), so absolute values vary a little with the machine. Memory

(container only, measured with Runtime after System.gc(), so approximate): the ArrayList needs about 4 bytes per ticket reference (≈ 390 KB for 100,000) while the HashMap needs about 38 bytes per entry (≈ 3,758 KB), roughly 10 times more. For n parked vehicles both are O(n) space, but the HashMap has the larger constant.

Structure	Find on exit	Duplicate check	Space (n parked)
Array / ArrayList	O(n)	O(n)	O(n)
HashMap	O(1) average	O(1) average	O(n), larger constant

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

A. Impact Analysis

What changed: two new vehicle categories – e-bike (flat fee) and delivery van (twice the car rate from 9 AM to 5 PM); a new one-day pass for visitors; the facility grows from 3 to 5 floors with a different number of slots on each floor.

Part of the design	Impact
Vehicle hierarchy	Two new subclasses (EBike, DeliveryVan). TwoWheeler, Car and ElectricCar are unaffected.
calculateFee()	The van's fee depends on the time of day, but the old method receives only minutes. A new overload calculateFee(LocalTime entry, int minutes) is added; the old methods remain.
Pass handling	A new OneDayPass joins MonthlyPass. Because ParkingService already uses the Pass interface (covers(date)), no if-checks on pass types are needed.
VehicleFactory	Two more cases (“ebike”, “van”).
Facility array	No change of type: the jagged array already allows a different number of slots per floor; only the configuration changes, e.g. {2,3,1,2,1} or {20,35,50,40,25}. A rectangular array or the formula floor × slotsPerFloor + slot would have had to be redesigned.
ParkingService	exit() calls the time-aware fee method (one line).

Classes that must be modified rather than extended: Vehicle (one new overload added, no existing method changed), VehicleFactory (two new cases) and ParkingService (exit uses the time-aware overload). No subclass or test of the old types changes.

B. Evaluation of the Original Design

- **Can new vehicle types be added only by writing subclasses?** Yes – EBike and DeliveryVan are new classes that override only what differs; nothing else has to know about them (polymorphism).
- **Are the pricing rules spread over many classes or in one place?** Partly spread: each rate lives in its subclass and the time-of-day rule in DeliveryVan. This is acceptable for a few types; with many rules a central PricingRule abstraction would be cleaner (see trade-offs).
- **Can the existing tested classes remain unchanged?** Yes: TwoWheeler, Car, ElectricCar, ParkingFacility, ParkingTicket and Chargeable are untouched; only the three small changes in A are needed.

Conclusion: the original design is appropriate and only needs to be modified, not redesigned.

C. Adaptation (decision: MODIFY / extend)

Revised structure: Vehicle gains the overload `calculateFee(LocalTime entry, int minutes)` whose default calls the old method; EBike overrides `calculateFee(int)` with a flat Rs 20; DeliveryVan extends Car and overrides the time-aware overload to multiply by 2 for entry from 09:00 up to (not including) 17:00; Pass stays an interface with MonthlyPass and the new OneDayPass; ParkingFacility is unchanged. The code below is taken from the executed source (Task12/Main.java):

```
// Added to Vehicle (existing methods unchanged)
public double calculateFee(LocalTime entry, int minutes) { return calculateFee(minutes); }
class EBike extends Vehicle {
    public static final double FLAT_FEE = 20;
    public EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 0; }
    @Override public double calculateFee(int minutes) {
        billableHours(minutes);                 // only validates the duration
        return FLAT_FEE;                        // flat fee per visit
    }
}
class DeliveryVan extends Car {
    public DeliveryVan(String regNo) { super(regNo); }
    @Override public double calculateFee(LocalTime entry, int minutes) {
        boolean peak = !entry.isBefore(LocalTime.of(9, 0)) && entry.isBefore(LocalTime.of(17,
0));
        return calculateFee(minutes) * (peak ? 2 : 1);      // double rate 09:00-17:00
    }
}
```

```
class OneDayPass implements Pass {
    public static final double PRICE = 100;
    private final LocalDate day;
    public OneDayPass(LocalDate day) {
        if (day == null) throw new IllegalArgumentException("Pass date required");
        this.day = day;
    }
    @Override public boolean covers(LocalDate d) { return d.equals(day); }
    public double getPrice() { return PRICE; }
}
// VehicleFactory: two new cases
case "ebike": return new EBike(regNo);
case "van":   return new DeliveryVan(regNo);
// ParkingService.exit(): v.calculateFee(minutes)  ->  v.calculateFee(t.getEntry(), minutes)
// Facility with 5 floors of different size
new ParkingFacility(new int[]{2, 3, 1, 2, 1});
```

Test results for the new categories (program output, Task12/output_new.txt):

No.	Test	Expected	Actual	Result
N1	E-bike, 300 min (flat fee)	20.0	20.0	Pass
N2	Van 10:00, 90 min (peak)	120.0	120.0	Pass
N3	Van 18:00, 90 min (off-peak)	60.0	60.0	Pass
N4	Van exactly 09:00 (boundary)	120.0	120.0	Pass
N5	Van exactly 17:00 (boundary)	60.0	60.0	Pass
N6	Car, valid one-day pass, 600 min	0.0	0.0	Pass
N7	Car, expired one-day pass	240.0	240.0	Pass
N8	One-day pass price	100.0	100.0	Pass
N9	E-bike, negative duration	IAE: Duration cannot be negative	IAE: Duration cannot be negative	Pass
N10	Facility {2,3,1,2,1}: 9 slots	9 allocated, 10th full	9 allocated, 10th full	Pass
N11	Facility {20,35,50,40,25}	170	170	Pass

```
DAY-4 CHALLENGE - NEW CATEGORIES AND ONE-DAY PASS
-------------------------------------------------
No.  Test case                      Expected                          Actual                            Result
-------------------------------------------------------------------------------------------------------------------
N1   E-bike, 300 min (flat fee)     20.0                              20.0                              Pass
N2   Van 10:00, 90 min (peak)       120.0                             120.0                             Pass
N3   Van 18:00, 90 min (off-peak)   60.0                              60.0                              Pass
N4   Van exactly 09:00 (boundary)   120.0                             120.0                             Pass
N5   Van exactly 17:00 (boundary)   60.0                              60.0                              Pass
N6   Car, valid one-day pass, 600 min  0.0                             0.0                               Pass
N7   Car, expired one-day pass      240.0                             240.0                             Pass
N8   One-day pass price             100.0                             100.0                             Pass
N9   E-bike, negative duration      IAE: Duration cannot be negative  IAE: Duration cannot be negative  Pass
N10  Facility {2,3,1,2,1}: 9 slots  9 allocated, 10th full            9 allocated, 10th full            Pass
N11  Facility {20,35,50,40,25}      170                               170                               Pass

New-category tests: 11 passed, 0 failed

$
```

Regression run – the complete Task 12 program re-runs all 21 original tests and the 11 new ones:



D. Evidence-Based Defence

- **Existing classes that had to change: 3** – Vehicle (one overload added), VehicleFactory (2 cases), Parking Service (one line in exit()). Unchanged: TwoWheeler, Car, ElectricCar, ParkingTicket, ParkingFacility, Chargeable, Pass, MonthlyPass. New classes: EBike, DeliveryVan, OneDayPass.
- **Ease of adding a further type:** write one subclass and add one factory case – about 5 lines and no change to the fee loops or the facility. A type with a different pricing basis (e.g. weekend rates) only needs to override the time-aware method.
- **Test results:** all 11 new-category tests pass and all 21 original tests (Rs 330 scenario, boundaries, invalid cases) give the same results after the change – 32 of 32 pass, so there is no regression.
- **Trade-offs:** a full PricingRule strategy (one place for all prices, prices changeable at run time) would reduce pricing code spread over classes, but it is a bigger rewrite and adds an object to every vehicle; for six vehicle types the current approach is simpler. If the number of rules grows (discounts, events, surge pricing) a PricingRule abstraction would then be justified. The jagged array keeps O(1) slot access, but the scan in allocate() stays O(F × S) and could be improved with the stack approach of Task 3.